

Production Push Notifications

iOS Home Screen PWA Web Push + Android FCM — Server Implementation Guide

Implementation handoff document for a coding agent (e.g. Gemini). This guide focuses on a secure, server-driven architecture that does not depend on keeping the web app alive in memory.

Current official references checked: Apple Developer / WebKit and Firebase Cloud Messaging documentation. Apple Web Push is standards-based and does not require Apple Developer Program membership for Web Push. ■cite■turn0search0■turn0search1■

Scope: iOS PWA Web Push using VAPID + Android FCM using Firebase Admin SDK / FCM HTTP v1. The backend should support both behind one notification service.

1. Target Architecture

```
YOUR BACKEND | +-----+-----+ | | iOS PWA Android | | Web Push/VAPID FCM | | Apple push service
Google FCM | | +-----+-----+ | DEVICE | NOTIFICATION
```

The client application registers for notifications and sends its push subscription/token to the backend. The backend stores those credentials and is solely responsible for deciding when a notification is needed and sending it. The client must not rely on JavaScript timers, WebSockets, or staying resident in RAM for notification delivery.

Apple documents the server flow as: obtain a push subscription, register its endpoint and encryption keys with your push server, handle pushes with a service worker, and send notifications from the server. Apple says Web Push is supported by Home Screen web apps on iOS/iPadOS 16.4+ and does not require Apple Developer Program membership.

■cite■turn0search0■turn0search1■

Firebase recommends the Firebase Admin SDK for server-side FCM because it handles authorization and supports popular server languages; FCM HTTP v1 is the lower-level alternative. ■cite■turn0search2■turn0search3■

2. Non-Negotiable Security Rules

- **Never put private credentials in frontend JavaScript.** VAPID private key and Firebase service-account private key belong only on the trusted backend.
- **Never commit secrets to Git.** Add secret files to .gitignore and preferably keep secrets outside the repository.
- **Do not log secrets.** Never log VAPID private keys, service-account JSON, OAuth access tokens, FCM registration tokens, or full Web Push subscriptions.
- **Use HTTPS everywhere.** Production push registration and application traffic must use TLS.
- **Use least privilege.** Give the Firebase service account only the permissions required to send FCM messages.
- **Protect notification registration endpoints.** A user must only be able to register/update subscriptions for their own authenticated account.
- **Validate ownership.** Do not accept an arbitrary user_id from the client and attach it to another account's subscription.
- **Handle credential rotation.** Keep VAPID keys stable unless deliberately rotating them; rotate Firebase service-account credentials according to your operational policy.
- **Delete invalid subscriptions/tokens.** If the push provider reports an expired/invalid destination, disable or remove it instead of retrying forever.
- **Use retry with exponential backoff** for transient provider/network failures; do not retry permanent authentication or invalid-token errors blindly. Firebase explicitly calls for retry handling with exponential backoff. ■cite■turn0search2■

3. Database Design

3.1 Unified push_subscriptions table

```
CREATE TABLE push_subscriptions ( id UUID PRIMARY KEY, user_id UUID NOT NULL REFERENCES users(id) ON
DELETE CASCADE, platform TEXT NOT NULL CHECK (platform IN ('ios_web', 'android')), provider TEXT NOT NULL
CHECK (provider IN ('webpush', 'fcm')), endpoint TEXT, p256dh TEXT, auth TEXT, fcm_token TEXT, device_id
TEXT, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
last_success_at TIMESTAMPTZ, last_failure_at TIMESTAMPTZ, disabled_at TIMESTAMPTZ ); CREATE INDEX
idx_push_subscriptions_user ON push_subscriptions(user_id); CREATE INDEX idx_push_subscriptions_active ON
push_subscriptions(user_id) WHERE disabled_at IS NULL;
```

For iOS Web Push, store the subscription endpoint and encryption keys together. For Android, store the current FCM registration token. A user may have multiple rows because one account can have multiple devices/browsers. Apple explicitly instructs servers to store the push endpoint and encryption keys with the user's account. ■cite■turn0search0■

Treat FCM registration tokens as sensitive credentials. Firebase explicitly says registration tokens must be kept secret. ■cite■turn0search2■

4. iOS Home Screen PWA Web Push Setup

4.1 Preconditions

- Serve the production site over HTTPS.
- Provide a valid Web App Manifest and Service Worker.
- The user must add the website to the Home Screen / use it as a Home Screen web app for iOS Web Push.
- Request notification permission only from a direct user interaction, such as an explicit 'Enable notifications' button. Apple documents this requirement. ■cite■turn0search0■turn0search1■
- Use feature detection rather than browser/version sniffing.

4.2 Generate VAPID keys

VAPID is the server identity mechanism used for Web Push. Generate the key pair once on a trusted machine/server. With Node.js and the common web-push package:

```
npm install web-push npx web-push generate-vapid-keys
```

Record the resulting public and private keys. The **public key** is sent to the browser and used in `PushManager.subscribe()`. The **private key never leaves the backend**.

```
# Example environment variables VAPID_PUBLIC_KEY=... VAPID_PRIVATE_KEY=...
VAPID_SUBJECT=mailto:admin@example.com
```

Do not generate a new VAPID key pair on every deployment. The public key used by `PushManager.subscribe` must match the VAPID public key used when sending pushes. Apple documents a `VapidPkHashMismatch` error when they do not match. ■cite■turn0search0■

4.3 Service Worker

```
self.addEventListener('push', (event) => { const data = event.data ? event.data.json() : {};
event.waitUntil( self.registration.showNotification( data.title || 'Your App', { body: data.body || '',
icon: '/icons/icon-192.png', badge: '/icons/badge.png', data: { url: data.url || '/' } } ) )); });
self.addEventListener('notificationclick', (event) => { event.notification.close(); event.waitUntil(
clients.openWindow(event.notification.data?.url || '/') ); });
```

Apple says Safari does not support invisible push notifications: the service worker should present the notification immediately after receiving the push. ■cite■turn0search0■

4.4 Client subscription flow

```
const permission = await Notification.requestPermission(); if (permission !== 'granted') return; const
registration = await navigator.serviceWorker.ready; const subscription = await
registration.pushManager.subscribe({ userVisibleOnly: true, applicationServerKey: VAPID_PUBLIC_KEY }); //
POST this subscription to your authenticated backend. await fetch('/api/push/web-subscription', { method:
'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(subscription) });
```

The backend endpoint must derive the user identity from the authenticated session/token, not from a `user_id` supplied by the browser.

4.5 Backend Web Push sender

```
const webpush = require('web-push'); webpush.setVapidDetails( process.env.VAPID_SUBJECT,  
process.env.VAPID_PUBLIC_KEY, process.env.VAPID_PRIVATE_KEY ); await webpush.sendNotification(  
subscription, JSON.stringify({ title: 'DayByDay', body: 'Time to log today's activity', url:  
'https://example.com/' })), { TTL: 3600, urgency: 'high' } );
```

Apple documents TTL, Authorization/VAPID, Content-Encoding, optional Topic, and Urgency headers. A TTL tells the push service how long it may retain an undelivered notification; Apple states storage may be up to 30 days or less depending on TTL, subject to push-service storage limits. The payload limit is 4 KB. ■cite■turn0search0■

4.6 iOS provider errors

- 201: accepted/success.
- 400: malformed request.
- 403: authentication problem.
- 404: invalid endpoint/path.
- 410: device token/endpoint expired — disable/remove the subscription.
- 413: payload too large.
- 429: too many requests — back off.
- 500/503: provider/server issue — retry with backoff where appropriate.
- VapidPkhHashMismatch: subscription's public VAPID key differs from server's VAPID key.
- BadJwtToken / BadAuthorizationHeader: inspect VAPID signing/audience/expiration and credentials.

Apple provides detailed HTTP status and reason codes and expects the sender to inspect responses.

■cite■turn0search0■

5. Android FCM Setup

5.1 Create/configure Firebase project

- Create or use a Firebase project for the Android application.
- Register the Android application with its exact application/package ID.
- Add the Firebase configuration required by the Android client.
- Add Firebase Cloud Messaging to the Android app.
- For Android 13+, request POST_NOTIFICATIONS runtime permission. Firebase documents that notifications cannot be shown until permission is granted. ■cite■turn0search4■

5.2 Enable FCM HTTP v1

Use the modern FCM HTTP v1 API and/or Firebase Admin SDK. Firebase recommends the Admin SDK for server environments. Enable the Firebase Cloud Messaging API (V1) in the project's Cloud Messaging settings if required.

■cite■turn0search2■

5.3 Create server credentials

- Firebase Console → Project Settings → Service Accounts.
- Generate a new private key for the service account if your non-Google server needs a service-account JSON credential.
- Download the JSON once and secure it immediately.
- Do not commit the JSON to GitHub or package it into the Android app.
- Prefer GOOGLE_APPLICATION_CREDENTIALS pointing to the credential file, or another secure secret-management mechanism. Firebase recommends the environment-variable/ADC approach over hard-coding credentials.

■cite■turn0search3■turn0search5■

```
# Example on the Oracle Linux server export
GOOGLE_APPLICATION_CREDENTIALS=/etc/daybyday/secrets/firebase-service-account.json
```

If possible, restrict filesystem permissions so only the service account running the backend can read the credential file.

5.4 Backend Firebase Admin SDK

```
npm install firebase-admin

const { initializeApp, applicationDefault } = require('firebase-admin/app'); const { getMessaging } =
require('firebase-admin/messaging'); initializeApp({ credential: applicationDefault() }); const messaging
= getMessaging(); await messaging.send({ token: fcmToken, notification: { title: 'DayByDay', body: 'Time
to log today's activity' }, data: { url: 'https://example.com/' } });
```

Firebase documents that the Admin SDK can send to individual app instances and is the recommended server-side option for common server environments. ■cite■turn0search2■

5.5 Token lifecycle

- Obtain the latest FCM registration token on the Android client.
- POST the token to your authenticated backend.
- Upsert by user + device/app instance as appropriate.
- Update the token whenever Firebase reports a token change.
- If FCM reports an invalid/unregistered token, mark it disabled and stop sending to it.
- Never assume an FCM token is permanent.

Firebase explicitly notes that registration tokens can change and recommends retrieving the latest identifier and sending it to the app server when it changes. ■cite■turn0search4■

6. Unified Notification Service

```
sendNotification(userId, payload): subscriptions = loadActiveSubscriptions(userId) for subscription in
subscriptions: if subscription.provider == 'webpush': sendWebPush(subscription, payload) if
subscription.provider == 'fcm': sendFCM(subscription, payload) record result update last_success_at /
last_failure_at disable permanent-invalid subscriptions
```

The application business logic should call only this abstraction. It should not contain provider-specific push code throughout the application.

7. Notification Scheduling

Do not use client-side setTimeout/setInterval as the source of truth for scheduled notifications. Use a server-side scheduler/worker.

```
notification_jobs ----- id user_id type payload scheduled_for status attempts last_error
created_at sent_at
```

A worker periodically claims due jobs using a database-safe mechanism, sends the notification, and records the result. Make job claiming idempotent so two workers cannot accidentally send duplicates.

For a small deployment on an existing Oracle VM, a persistent worker or cron-triggered worker is sufficient. The important property is that the server—not the PWA—owns the schedule.

8. Reliability & Retry Policy

- **Permanent errors:** invalid/expired token, malformed credentials, mismatched VAPID key. Fix/disable; do not blindly retry.
- **Transient errors:** network timeout, HTTP 5xx, provider temporary unavailability. Retry with exponential backoff and jitter.
- **Rate limiting:** respect 429 responses and back off.

- **Idempotency:** assign a unique notification/job ID and prevent duplicate sends caused by worker retries.
- **Observability:** record provider, subscription ID, attempt number, timestamp, result category, and provider error code—but never secrets.
- **Dead-letter handling:** after a bounded number of failed attempts, move the job to a failed/dead-letter state for inspection.

Firebase explicitly requires the server environment to be able to handle requests and resend using exponential backoff. Apple likewise provides provider error responses that should drive corrective action. ■cite■turn0search2■turn0search0■

9. Credential & Secret Storage Checklist

- VAPID_PRIVATE_KEY → server environment/secret store only.
- Firebase service-account JSON private_key → server filesystem with restrictive permissions or a secret manager.
- Never put either credential in React/Next.js public environment variables.
- Never use NEXT_PUBLIC_* / VITE_* / client-exposed variables for private credentials.
- Never print environment variables in logs or diagnostics.
- Never send service-account JSON to the mobile app.
- Keep backups of secrets encrypted and access-controlled.
- If a private key is exposed, rotate/revoke it immediately and replace it in the server.
- Use a dedicated production Firebase service account rather than a personal Google account credential.

10. Network & Firewall

Apple requires the server to be able to reach push.apple.com endpoints. Apple documents allowing https://*.push.apple.com when outbound filtering is in place, and says the service should maintain TLS-encrypted connections. ■cite■turn0search0■turn0search1■

- Allow outbound HTTPS (TCP 443) from the backend to required push-provider endpoints.
- Do not expose internal secret files or database ports to the public internet.
- Keep PostgreSQL bound to the required interfaces only and protected by authentication/firewall rules.
- Use HTTPS for every client-to-backend subscription registration request.

11. Testing Plan

- Test with PWA open and foreground.
- Test with PWA backgrounded.
- Test after leaving the PWA unused for a long period.
- Test after the PWA process is no longer resident.
- Test device locked.
- Test device offline, then reconnect.
- Test notification permission denied/granted.
- Test iOS Focus/notification settings as expected user-side conditions.
- Test Android 13+ notification permission.
- Test Android token rotation.
- Test invalid/deleted subscriptions.
- Test provider 5xx/429 retry behavior.

- Test duplicate worker execution and ensure idempotency.
- Test secret exposure controls and confirm secrets never appear in client bundles or logs.

12. Definition of Done

- iOS Home Screen PWA can request notification permission from a user gesture.
- iOS push subscription is registered with authenticated backend.
- Backend stores endpoint + encryption keys.
- VAPID public/private key pair is generated once and securely stored.
- iOS push works while the PWA is not open.
- Android obtains and refreshes an FCM registration token.
- Backend stores FCM token securely.
- Firebase Admin SDK / HTTP v1 is configured with server-only credentials.
- Android notification permission is handled on Android 13+.
- A unified notification service sends to both providers.
- Scheduled notifications are triggered by the server worker, not client timers.
- Invalid destinations are disabled/removed.
- Transient failures retry with exponential backoff.
- Notification jobs are idempotent.
- Secrets are absent from source control, client bundles, and logs.
- Production monitoring records success/failure without leaking credentials.

13. Recommended Environment Variables

```
# Web Push VAPID_PUBLIC_KEY=... VAPID_PRIVATE_KEY=... VAPID_SUBJECT=mailto:admin@example.com # Firebase
GOOGLE_APPLICATION_CREDENTIALS=/etc/daybyday/secrets/firebase-service-account.json # App
APP_BASE_URL=https://your-domain.example DATABASE_URL=...
```

Never paste real production values into this document or into source code. Populate them only in the deployment environment.

14. Important Provider Facts

Apple Web Push: Home Screen web apps on iOS/iPadOS 16.4+ can use standards-based Web Push; Apple says no Apple Developer Program membership is required for Web Push. [■cite■turn0search0■turn0search1■](#)

Apple's push server requirements include VAPID, encrypted payloads, RFC 8030 Web Push packaging, TLS, SNI, and access to *.push.apple.com. Apple documents a 4 KB payload limit and response codes/reasons for troubleshooting. [■cite■turn0search0■](#)

Firebase FCM: Firebase recommends the Admin SDK for server-side sending. Service-account credentials and FCM registration tokens must be protected; OAuth access tokens used by HTTP v1 are short-lived. [■cite■turn0search2■turn0search3■](#)

15. Implementation Instruction for Coding Agent

Implement this in phases and do not break existing notification behavior until the new path is verified:

- **Phase 1:** inspect the existing frontend, backend, authentication, database, scheduler, and current notification implementation. Do not assume the framework.
- **Phase 2:** implement the unified push_subscriptions schema and authenticated registration endpoints.

- **Phase 3:** implement iOS Web Push + VAPID + Service Worker.
- **Phase 4:** implement Android FCM token registration + Firebase Admin SDK sender.
- **Phase 5:** implement the unified notification service and server-side scheduler/worker.
- **Phase 6:** implement provider error classification, retries, invalid-token cleanup, idempotency, and logging.
- **Phase 7:** run the full testing plan on physical iOS and Android devices.
- **Phase 8:** verify that no secret is shipped to the frontend, committed to Git, or written to logs.
- **Phase 9:** document deployment environment variables and rollback steps.

16. Do Not Implement These Anti-Patterns

- Do not keep the PWA alive in memory to receive notifications.
- Do not rely on setInterval/setTimeout for notifications.
- Do not poll the backend from the PWA as the notification mechanism.
- Do not put VAPID private keys in frontend code.
- Do not put Firebase service-account JSON in the Android app.
- Do not send FCM using deprecated legacy server keys when HTTP v1/Admin SDK is available.
- Do not trust a client-supplied user_id when registering a subscription.
- Do not retry permanently invalid subscriptions forever.
- Do not silently swallow provider errors.

17. Official Documentation

Apple — Sending web push notifications in web apps and browsers:

<https://developer.apple.com/documentation/usernotifications/sending-web-push-notifications-in-web-apps-and-browsers>

■cite■turn0search0■

WebKit — Web Push for Web Apps on iOS and iPadOS:

<https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/> ■cite■turn0search1■

Firebase — FCM server environment: <https://firebase.google.com/docs/cloud-messaging/server-environment>

■cite■turn0search2■

Firebase — FCM HTTP v1: <https://firebase.google.com/docs/cloud-messaging/send/v1-api> ■cite■turn0search3■

Firebase — Android FCM setup: <https://firebase.google.com/docs/cloud-messaging/android/get-started>

■cite■turn0search4■

Firebase — Admin SDK setup: <https://firebase.google.com/docs/admin/setup> ■cite■turn0search5■